BUSINESS REQUIREMENTS DOCUMENT

ShopMe — E-Commerce Platform

End-to-End BA Analysis: Authentication, Catalog, Cart, Order Lifecycle & Admin Workflows

Prepared by: Abhishek Chebolu | Technical Business Analyst | 2025

Field	Details
Document Title	ShopMe E-Commerce Platform — Business Requirements Document
Version	1.0
Status	Approved for Review
Prepared By	Abhishek Chebolu, Technical Business Analyst
Date	2025
Platform Type	Monolith Architecture — Spring Boot
Domain	E-Commerce / Retail / Product Platform
Tech Stack	Java, Spring Boot, Spring Security, OAuth 2.0, Thymeleaf, MySQL, Amazon S3
GitHub Reference	github.com/abhichebolu/shopMe-Ecommerce-Website-abhi

DOCUMENT PURPOSE

This BRD documents the complete business and functional requirements for ShopMe, a Spring Boot-based e-commerce platform serving both admin and customer personas. It covers system architecture context, module-wise workflow analysis, order lifecycle state modeling, API interaction flows, role-based access control, failure scenarios, user stories, and UAT strategy.

This document is part of a BA portfolio project demonstrating applied requirement engineering on a real open-source e-commerce system.

1. EXECUTIVE SUMMARY

ShopMe is a full-featured e-commerce web application built on Spring Boot using a Monolith architecture. It serves two primary user groups: Administrators who manage the product catalog, orders, and users, and Customers who browse products, place orders, and track purchases.

The platform supports multi-role admin access (Admin, Editor, Salesperson, Shipper, Assistant), OAuth 2.0-based social login, product image storage via Amazon S3, internationalization (English and Turkish), and a complete order-to-payment lifecycle for customers.

This BRD analyzes ShopMe from a Technical Business Analyst perspective — documenting system workflows, API interactions, role-based access rules, order state transitions, failure scenarios, and acceptance criteria that would govern real-world requirement delivery.

Module	Primary Persona	Core Function
Authentication	Customer + Admin	Registration, Login, OAuth 2.0 (Google/GitHub), Spring Security
Product Catalog	Admin + Customer	CRUD for products, categories, images via Amazon S3
Cart Management	Customer	Add/remove items, quantity update, stock validation
Order Processing	Customer + Admin	Checkout, payment, order state lifecycle management
Admin Dashboard	Admin roles	Role-based access: manage users, products, orders, reports
User Management	Admin	Customer and admin account management, role assignment
File / Image Mgmt	Admin	Product image upload/delete via Amazon S3 Bucket
i18n / Localization	Admin	Turkish and English language support for admin interface

2. SCOPE

2.1 In Scope

- Customer registration, login, and OAuth 2.0 authentication flow
- Product catalog management — category and product CRUD by admin roles
- Amazon S3 image upload, retrieval, and deletion for product images
- Customer cart management — add, update, remove items with stock validation
- End-to-end order lifecycle: cart → checkout → payment → order confirmation → shipping → delivery
- Role-based admin access control: Admin, Editor, Salesperson, Shipper, Assistant
- Order monitoring and status update by admin roles
- Internationalization (i18n) for Turkish and English in admin interface
- CSV export of product/order data (Apache POI / SuperCSV integration)

2.2 Out of Scope

- Mobile application (iOS / Android) — web only
- Third-party logistics (3PL) API integration
- Real payment gateway integration (payment simulated in current implementation)
- Customer loyalty / rewards program
- Multi-vendor marketplace functionality
- Real-time inventory sync with physical warehouse

3. STAKEHOLDERS & USER ROLES

Role	Type	Access Level	Key Responsibilities
Admin	Internal	Full Access	All platform functions: users, products, orders, settings
Editor	Internal	Catalog Only	Create/edit/delete categories and products, manage images
Salesperson	Internal	Orders + Customers	View/update orders, manage customer accounts, view products
Shipper	Internal	Shipping Only	View orders, update shipping status and tracking
Assistant	Internal	Read Only (partial)	View products and orders, no create/delete permissions
Customer	External	Customer Portal	Register, browse, cart, checkout, track orders
Product Owner	Internal	Stakeholder	Defines feature priorities, approves requirements
Tech Lead	Internal	Stakeholder	Reviews technical feasibility of requirements
BA (Author)	Internal	Requirements Owner	Documents requirements, facilitates UAT sign-off

4. SYSTEM ARCHITECTURE CONTEXT

ShopMe is built as a Monolith architecture using Spring Boot. All modules (authentication, catalog, cart, orders, admin) run within a single deployable application. This is significant from a BA perspective because all inter-module communication is internal method calls — not API calls between microservices.

ARCHITECTURE OVERVIEW

Frontend: Thymeleaf templates (server-side rendering) + HTML/CSS/JavaScript
Backend: Java + Spring Boot (Monolith) + Spring MVC
Security: Spring Security + OAuth 2.0 (Google / GitHub login)
Database: MySQL (relational) — JPA/Hibernate ORM
File Storage: Amazon S3 Bucket (product images)
Export: Apache POI (Excel) + SuperCSV (CSV exports)
i18n: Spring i18n for Turkish and English admin interface
Testing: JUnit for unit testing

BA IMPLICATION: MONOLITH vs MICROSERVICES

In a monolith architecture, requirements for cross-module behavior (e.g., order creation triggering inventory update) must specify internal business rules rather than API contracts between services. The BA must document data flow and business logic rules with the same rigor as API contracts in microservices — because the same failure scenarios (race conditions, data consistency) still apply.

5. MODULE WORKFLOWS

5.1 Authentication Module

ShopMe supports two authentication paths: standard registration/login using Spring Security, and social login via OAuth 2.0 (Google/GitHub).

STANDARD REGISTRATION FLOW

Step 1: Customer visits /register, fills name, email, password
Step 2: System validates email format and checks for duplicate email in DB
Step 3: Password hashed using BCrypt before storage (never stored as plaintext)
Step 4: Verification email sent to customer (if email verification enabled)

Step 5: Customer clicks verification link → account activated
Step 6: Customer redirected to login page
Failure: Duplicate email → error message shown, registration blocked

OAUTH 2.0 LOGIN FLOW (Google / GitHub)

Step 1: Customer clicks 'Login with Google' or 'Login with GitHub'
Step 2: System redirects to OAuth provider authorization endpoint
Step 3: Customer authenticates with provider and grants permission
Step 4: Provider returns authorization code to ShopMe callback URL
Step 5: Spring Security exchanges code for access token
Step 6: System retrieves customer email and name from provider
Step 7a: If email exists in DB → login successful, session created
Step 7b: If email new → auto-register customer, then login
Step 8: Customer redirected to home page or last visited page

ADMIN LOGIN FLOW

Step 1: Admin navigates to /admin/login
Step 2: Enters credentials (username/email + password)
Step 3: Spring Security validates credentials against admin_users table
Step 4: On success → session created with role authority loaded
Step 5: Role determines dashboard view and accessible menu items
Step 6: On failure → error message, account locked after 5 failed attempts

5.2 Product Catalog Module

The catalog module manages two hierarchical entities: Categories and Products. Categories are top-level groupings. Products belong to a category and contain images stored in Amazon S3.

CATEGORY MANAGEMENT FLOW (Admin / Editor)

Create: Admin/Editor fills category name, alias, description, parent category (optional)
  System checks for duplicate alias → error if exists
  Category saved to DB, available immediately in product creation
Edit: Admin/Editor updates fields → system re-validates alias uniqueness
Delete: System checks if category has associated products
  If products exist → delete blocked, user shown warning
  If no products → category deleted from DB
Export: Admin can export category list to CSV via SuperCSV

PRODUCT MANAGEMENT FLOW (Admin / Editor)

Create: Editor fills product name, alias, category, price, cost, stock quantity,
  dimensions, description, and enabled/disabled status
Images: Editor uploads up to 5 product images
  Images sent to Amazon S3 via Amazon SDK
  S3 returns image URL stored in DB linked to product
  Main image designated as primary display image
Edit: Any field can be updated; images can be replaced (old S3 object deleted)
Delete: Product soft-disabled or hard-deleted
  If product exists in active order → hard delete blocked
  All S3 images deleted on hard delete
Export: Product list exportable to Excel (Apache POI) or CSV (SuperCSV)

5.3 Cart Module

The cart is session/DB-persisted for logged-in customers. Guest carts are session-only and converted to persistent cart on login.

CART WORKFLOW

Add Item: Customer clicks 'Add to Cart' on product page
System checks product stock quantity
If in stock → item added to cart with quantity 1
If out of stock → button disabled, 'Out of Stock' shown
Update Qty: Customer changes quantity in cart
System re-validates stock → if requested qty > stock, capped at max available
Remove Item: Customer removes item → line item deleted from cart
Cart Total: Recalculated on every add/update/remove action
Applies any active discount/promotion if configured
Proceed: Customer proceeds to checkout → stock re-validated at this point
If stock changed since add (race condition) → customer notified

RACE CONDITION: STOCK VALIDATION AT CHECKOUT

Scenario: Customer A and Customer B both add last unit of Product X to cart simultaneously
Problem: Both see 'In Stock' at add-to-cart time
Solution: Final stock check happens at checkout (order creation), not at add-to-cart
Outcome: Whichever customer completes checkout first gets the item
Second customer receives: 'Sorry, Product X is no longer available in requested quantity'
BA Note: This scenario must be covered in UAT test cases TC-012 and TC-013

5.4 Order Lifecycle Module

The order module manages the complete lifecycle from checkout through delivery. Order states are updated by different admin roles based on their access level.

Order State	Description	Who Updates / Next State
NEW	Order placed by customer. Payment pending or initiated.	System auto-set on checkout → PROCESSING
PROCESSING	Payment confirmed. Order being prepared for dispatch.	Admin/Salesperson → PACKAGED
PACKAGED	Items packed and ready for pickup by shipper.	Admin/Salesperson → PICKED
PICKED	Shipper has picked up the package.	Shipper → SHIPPING
SHIPPING	Package in transit to customer address.	Shipper → DELIVERED
DELIVERED	Package delivered to customer. Order complete.	Shipper marks delivered → terminal state
RETURN_REQUESTED	Customer has requested a return/refund.	Admin/Salesperson → RETURNED or REJECTED
RETURNED	Return approved. Refund initiated.	Admin → terminal state, triggers refund
CANCELLED	Order cancelled before shipping.	Customer (before PICKED) / Admin anytime

5.5 Checkout Flow

COMPLETE CHECKOUT SEQUENCE

```
Step 1:  Customer clicks 'Proceed to Checkout' from cart page
Step 2:  System re-validates all cart items against current stock
         If any item out of stock → customer notified, item flagged in cart
Step 3:  Customer confirms/adds shipping address
Step 4:  Customer selects shipping method (standard / express if available)
Step 5:  Order summary displayed with itemized total + shipping cost + tax
Step 6:  Customer clicks 'Place Order' → payment flow initiated
Step 7:  Payment processed (gateway integration / COD)
Step 8:  On payment success → order created in DB with status: NEW
         Stock decremented for all ordered items
         Order confirmation email sent to customer
Step 9:  Customer redirected to Order Confirmation page with order ID
Failure: Payment failure → order NOT created, stock NOT decremented
         Customer shown payment failure message with retry option
```

6. ROLE-BASED ACCESS CONTROL (RBAC)

Spring Security manages role-based access. Each admin role has a defined set of permitted actions. The BA must document these clearly to avoid scope creep during development and to create accurate UAT test cases.

Feature / Action	Admin	Editor	Salesperson	Shipper	Assistant
View product catalog	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes
Create / Edit product	✓ Yes	✓ Yes	✗ No	✗ No	✗ No
Delete product	✓ Yes	✓ Yes	✗ No	✗ No	✗ No
Create / Edit category	✓ Yes	✓ Yes	✗ No	✗ No	✗ No
Upload images to S3	✓ Yes	✓ Yes	✗ No	✗ No	✗ No
View all orders	✓ Yes	✗ No	✓ Yes	✓ Yes	✓ Yes
Update order status	✓ Yes	✗ No	✓ Yes	Ship only	✗ No
Manage customer accounts	✓ Yes	✗ No	✓ Yes	✗ No	✗ No
Manage admin users	✓ Yes	✗ No	✗ No	✗ No	✗ No
Export data (CSV/Excel)	✓ Yes	✓ Yes	✓ Yes	✗ No	✗ No
View sales reports	✓ Yes	✗ No	✓ Yes	✗ No	✗ No
Change i18n language	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes

7. FUNCTIONAL REQUIREMENTS

7.1 Authentication

Req ID	Requirement	Priority
FR-001	System shall support customer self-registration with email, name, and password	HIGH
FR-002	System shall validate email format and reject duplicate email addresses at registration	HIGH
FR-003	System shall hash all passwords using BCrypt before storage — plaintext never stored or logged	HIGH
FR-004	System shall support OAuth 2.0 login via Google and GitHub using Spring Security	HIGH
FR-005	System shall auto-create a customer account on first OAuth login using provider email	HIGH
FR-006	System shall lock customer account after 5 consecutive failed login attempts	MEDIUM

Req ID	Requirement	Priority
FR-007	System shall maintain separate login paths for admin (/admin/login) and customer (/login)	HIGH
FR-008	System shall load role-based authorities into session immediately on admin login	HIGH

7.2 Product Catalog

Req ID	Requirement	Priority
FR-010	System shall support hierarchical category structure (parent → child categories)	HIGH
FR-011	System shall prevent deletion of categories that have associated active products	HIGH
FR-012	System shall support up to 5 product images per product, stored in Amazon S3	HIGH
FR-013	System shall delete S3 image objects when a product image is replaced or product is hard-deleted	HIGH
FR-014	System shall allow admin to enable/disable products — disabled products hidden from customers	HIGH
FR-015	System shall support product alias (URL-friendly slug) with uniqueness validation	MEDIUM
FR-016	System shall support CSV and Excel export of product and category lists	MEDIUM

7.3 Cart & Order

Req ID	Requirement	Priority
FR-020	System shall validate product stock at add-to-cart and again at checkout (double validation)	HIGH
FR-021	System shall cap cart quantity at available stock — customer notified if requested qty exceeds stock	HIGH
FR-022	System shall decrement product stock only on successful payment and order creation	HIGH
FR-023	System shall NOT create an order record if payment fails — no stock decrement on payment failure	HIGH
FR-024	System shall send order confirmation email to customer on successful order placement	HIGH
FR-025	System shall allow order cancellation by customer only before order reaches PICKED state	HIGH
FR-026	System shall allow order cancellation by Admin at any state	HIGH
FR-027	System shall restore product stock when an order is cancelled or returned	HIGH
FR-028	Shipper role shall only be permitted to update orders in PICKED or SHIPPING state	HIGH

7.4 Non-Functional Requirements

Category	Requirement	Target
Performance	Product catalog page load time	< 2 seconds (P95)
Performance	Cart update response time	< 1 second
Security	Password storage	BCrypt hash, never plaintext
Security	Admin routes protected	All /admin/** require authenticated session with role
Security	S3 bucket access	Private bucket, pre-signed URLs for image access

Category	Requirement	Target
Availability	Application uptime	99.5% (non-critical e-commerce platform)
Scalability	Concurrent user sessions supported	500 concurrent users (monolith baseline)
Data Integrity	Stock decrement atomicity	DB transaction ensures stock never goes negative
i18n	Admin interface language support	English and Turkish, switchable per session
Export	Product/order export file generation time	< 5 seconds for up to 10,000 records

8. CRITICAL FAILURE SCENARIOS

SCENARIO 1: Payment Failure — Order Must NOT Be Created

Trigger: Customer completes checkout and payment gateway returns failure response
Risk: If order is created before payment confirmation, stock gets decremented incorrectly
Correct: Order creation and stock decrement must happen ONLY after payment success callback
System: Payment gateway calls back to /order/payment-success or /order/payment-failed
On failure: No order record created, no stock change, customer shown retry option
BA Note: This is a critical transactional boundary — must be documented in FR-023

SCENARIO 2: Cart-to-Checkout Race Condition

Trigger: Two customers add the last unit of Product X to cart simultaneously
Problem: Both pass add-to-cart stock check (stock = 1 at time of check for both)
Solution: Checkout performs a database-level stock lock before order creation
Outcome: First customer to checkout gets the item, stock decremented to 0
Second customer at checkout receives: out-of-stock notification
Second customer's cart updated to show item as unavailable
BA Note: Requires DB transaction with SELECT FOR UPDATE or equivalent lock

SCENARIO 3: S3 Image Upload Failure

Trigger: Admin uploads product image but Amazon S3 returns error (timeout, auth failure)
Risk: Product saved to DB without image URL, broken image displayed to customers
Correct: Image upload to S3 must succeed before product record is saved
On failure: Product save rolled back, admin shown: 'Image upload failed. Please retry.'
BA Note: S3 error codes to handle: 403 Forbidden, 503 Service Unavailable, timeout

SCENARIO 4: Order Cancellation After PICKED State

Trigger: Customer attempts to cancel order that is already in PICKED or SHIPPING state
System: Cancel button disabled/hidden for orders in PICKED, SHIPPING, DELIVERED states
Customer: Shown message: 'Order cannot be cancelled — already dispatched. Raise a return request.'
Admin: Can override and cancel at any state with mandatory cancellation reason logged
Stock: On admin-forced cancellation post-PICKED — stock restored after return confirmed

SCENARIO 5: OAuth Login — Email Mismatch / Provider Change

Trigger: Customer previously registered with email A@gmail.com using standard login
Now tries to login with Google OAuth using same email A@gmail.com

Expected: System detects existing email, links OAuth provider to existing account
Customer logged in successfully, no duplicate account created
Failure: If not handled, duplicate account created — customer has two separate profiles
BA Note: System must check email existence before creating new OAuth account (FR-005)

9. USER STORIES

Story ID	Persona	User Story	Priority
US-001	Customer	As a customer, I want to register with my email and password, so that I can place orders on ShopMe.	HIGH
US-002	Customer	As a customer, I want to login using my Google account, so that I don't need to remember a separate password.	HIGH
US-003	Customer	As a customer, I want to browse products by category, so that I can find items relevant to my needs quickly.	HIGH
US-004	Customer	As a customer, I want to add products to my cart with quantity selection, so that I can purchase multiple items in one order.	HIGH
US-005	Customer	As a customer, I want to see real-time stock availability on product pages, so that I know before adding to cart whether the item is available.	HIGH
US-006	Customer	As a customer, I want to proceed to checkout from my cart, so that I can review my order before placing it.	HIGH
US-007	Customer	As a customer, I want to receive an order confirmation email after placing an order, so that I have a record of my purchase.	HIGH
US-008	Customer	As a customer, I want to view my order history and current order status, so that I can track my purchases.	HIGH
US-009	Customer	As a customer, I want to cancel my order before it is dispatched, so that I can change my mind without penalty.	HIGH
US-010	Customer	As a customer, I want to raise a return request for a delivered order, so that I can get a refund for unsatisfactory items.	MEDIUM
US-011	Editor	As an editor, I want to create and manage product categories with parent-child hierarchy, so that the catalog is well-organized.	HIGH
US-012	Editor	As an editor, I want to upload up to 5 images per product to Amazon S3, so that customers see high-quality product visuals.	HIGH
US-013	Editor	As an editor, I want to enable or disable products without deleting them, so that I can temporarily hide items from the catalog.	HIGH
US-014	Salesperson	As a salesperson, I want to view all customer orders and update their status, so that I can manage the order fulfillment pipeline.	HIGH
US-015	Salesperson	As a salesperson, I want to export order reports to Excel, so that I can analyze sales performance offline.	MEDIUM
US-016	Shipper	As a shipper, I want to view orders assigned for shipping, so that I know which packages to pick up and deliver.	HIGH
US-017	Shipper	As a shipper, I want to update order status to SHIPPING and DELIVERED, so that the system reflects real-world delivery progress.	HIGH
US-018	Admin	As an admin, I want to manage all admin user accounts and assign roles, so that access control is maintained correctly.	HIGH
US-019	Admin	As an admin, I want to cancel any order at any stage with a logged reason, so that I can handle exceptional cases.	HIGH
US-020	Admin	As an admin, I want the admin interface to support both English and Turkish, so that team members can work in their preferred language.	MEDIUM

9.1 Acceptance Criteria — Sample Stories

US-002: OAuth 2.0 Login Acceptance Criteria

Given: Customer clicks 'Login with Google' on the login page
When: Customer completes Google authentication and grants permission
Then: System shall retrieve email and name from Google
And: If email exists in DB → customer logged in, redirected to home page
And: If email is new → customer account auto-created, then logged in
And: No duplicate account created if email already exists from standard registration
And: Session created with CUSTOMER role authority

US-007: Order Confirmation Email Acceptance Criteria

Given: Customer completes checkout and payment is successful
When: Order is created in DB with status NEW
Then: System shall send order confirmation email to customer's registered email
And: Email shall contain: Order ID, itemized product list, quantities, total amount, shipping address
And: Email shall be sent within 60 seconds of order creation
And: If email delivery fails → order still created, failure logged for retry

10. UAT TEST CASES

TC ID	Module	Scenario	Steps	Expected Result	Priority
TC-001	Auth	Successful standard registration	Enter valid name, email, password → submit	Account created. Success message shown. Login page displayed.	HIGH
TC-002	Auth	Duplicate email registration	Register with existing email	Error: 'Email already in use.' Registration blocked.	HIGH
TC-003	Auth	OAuth login — new customer	Click Google login → authenticate with new email	Account auto-created. Customer logged in. Redirected to home.	HIGH
TC-004	Auth	OAuth login — existing customer	Click Google login with email already in DB	Customer logged in. No duplicate account. Existing profile loaded.	HIGH
TC-005	Auth	Admin login — correct role loaded	Login as Shipper role	Dashboard shows shipping orders only. Product CRUD menu hidden.	HIGH
TC-006	Auth	Account locked after 5 failures	Enter wrong password 5 times	Account locked. Error message shown. Unlock email sent.	MEDIUM
TC-007	Catalog	Create product with S3 image	Editor creates product, uploads 1 image	Product saved. Image uploaded to S3. Image visible on product page.	HIGH
TC-008	Catalog	Delete category with active products	Admin deletes category with products	Error: 'Cannot delete — active products exist.' Category not deleted.	HIGH
TC-009	Catalog	Disable product	Editor disables active product	Product hidden from customer catalog. Still visible in admin.	HIGH

TC ID	Module	Scenario	Steps	Expected Result	Priority
TC-010	Catalog	S3 image upload failure	Simulate S3 timeout during upload	Product NOT saved. Error: 'Image upload failed.' Admin shown retry option.	HIGH
TC-011	Cart	Add in-stock item to cart	Customer adds Product A (stock: 5) to cart	Item added. Cart shows quantity 1. Stock not yet decremented.	HIGH
TC-012	Cart	Add out-of-stock item	Customer tries to add Product B (stock: 0)	'Add to Cart' button disabled. 'Out of Stock' label shown.	HIGH
TC-013	Cart	Race condition — last unit	Two customers checkout same last-stock item simultaneously	First to complete checkout succeeds. Second gets out-of-stock error.	HIGH
TC-014	Cart	Qty exceeds available stock	Customer sets cart quantity to 10 for item with stock 3	Quantity capped at 3. Message: 'Only 3 units available.'	HIGH
TC-015	Order	Successful checkout and order creation	Complete checkout with valid payment	Order created with status NEW. Stock decremented. Email sent.	HIGH
TC-016	Order	Payment failure — no order created	Simulate payment gateway failure at checkout	No order created. No stock change. Customer shown retry option.	HIGH
TC-017	Order	Customer cancels order before PICKED	Customer cancels order in PROCESSING state	Order status → CANCELLED. Stock restored. Refund initiated.	HIGH
TC-018	Order	Customer cannot cancel after PICKED	Customer attempts cancel on SHIPPED order	Cancel button hidden/disabled. Message: 'Raise a return request.'	HIGH
TC-019	Order	Shipper updates status	Shipper updates order from PICKED → SHIPPING	Status updated. Customer notified. Timestamp logged.	HIGH
TC-020	Order	Shipper cannot edit product	Shipper navigates to product catalog	Access denied. Redirected to dashboard with permission error.	HIGH
TC-021	Order	Admin cancels post-PICKED order	Admin cancels SHIPPED order with reason	Order CANCELLED. Reason logged. Stock restored after return confirmed.	MEDIUM
TC-022	Order	Return request raised by customer	Customer raises return on DELIVERED order	Return request created. Status: RETURN_REQUESTED. Admin notified.	MEDIUM
TC-023	Export	Product export to Excel	Salesperson exports product list	Excel file downloaded with all active product data. < 5 seconds.	MEDIUM
TC-024	i18n	Admin switches language to Turkish	Admin selects Turkish from language menu	All admin UI labels switch to Turkish. Data unchanged.	MEDIUM
TC-025	Security	Unauthorized admin route access	Logged-out user navigates to /admin/products	Redirected to /admin/login. Route not accessible without auth.	HIGH

11. DATA DICTIONARY

Entity / Field	Data Type	Format	Description	Example
customer_id	Integer	Auto-increment	Unique customer identifier	1001
customer_email	String	Max 128 chars	Customer email, used as login ID	abhi@gmail.com
password_hash	String	BCrypt 60 chars	BCrypt hashed password	$2a$10$...
oauth_provider	Enum	GOOGLE/GITHUB/LOCAL	Authentication provider	GOOGLE
category_alias	String	URL-safe, unique	URL slug for category	electronics
product_alias	String	URL-safe, unique	URL slug for product page	samsung-galaxy-s24
product_price	Decimal	10 digits, 2 dec	Customer-facing price in INR	29999.00
product_cost	Decimal	10 digits, 2 dec	Internal cost price	22000.00
stock_quantity	Integer	Non-negative	Available units in inventory	50
s3_image_url	String	Max 512 chars	Amazon S3 object URL for image	https://s3.aws.../img.jpg
order_id	Integer	Auto-increment	Unique order identifier	5001
order_status	Enum	Max 20 chars	Current order lifecycle state	PROCESSING
order_total	Decimal	10 digits, 2 dec	Total order value including tax	31500.00
admin_role	Enum	Max 20 chars	Admin user role for RBAC	EDITOR
cart_id	Integer	Auto-increment	Cart session identifier	2001

12. ASSUMPTIONS, CONSTRAINTS & SIGN-OFF

12.1 Assumptions

- Payment gateway integration is treated as a black box for this BRD — system behavior on success/failure callbacks is documented, not gateway internals.
- Amazon S3 bucket is pre-configured with correct IAM permissions — credential management is handled by infrastructure team, not application code.
- OAuth 2.0 provider credentials (Google/GitHub client ID and secret) are configured in application properties before deployment.
- Database transactions are managed by Spring's @Transactional annotation — rollback behavior on failure is assumed to work as per Spring default.
- Email delivery uses an SMTP service configured in application properties — email failure does not block order creation.

12.2 Constraints

- Monolith architecture limits horizontal scalability — concurrent user load is bounded by single-instance capacity.
- Admin interface i18n is limited to Turkish and English — additional languages require new message property files.
- Product images limited to 5 per product — enforced at application level before S3 upload.
- Order cancellation by customer is only permitted before PICKED state — post-dispatch requires return request workflow.

12.3 Sign-Off Table

Role	Name	Signature	Date
Technical Business Analyst	Abhishek Chebolu	_____	_____
Product Manager	_____	_____	_____
Engineering Lead	_____	_____	_____
QA Lead	_____	_____	_____
Security / Compliance	_____	_____	_____

PREPARED BY
Abhishek Chebolu | Technical Business Analyst
Email: abhichebolu@gmail.com | Location: Hyderabad, India
GitHub: github.com/abhichebolu | LinkedIn: [Your LinkedIn URL]
This document is part of a BA portfolio project demonstrating applied requirement engineering on the ShopMe open-source e-commerce platform (Spring Boot / Java).